

SMART CONTRACT SECURITY AUDIT

ThunderLoan Protocol Audit Report

A comprehensive security review of the ThunderLoan flash-loan protocol — combining static analysis, adversarial exploit testing, and 8 simulated attack vectors.

Prepared by Chijulybuilds

BLOCKCHAIN SECURITY RESEARCHER

October 2026 · Slither · Aderyn · Foundry

ThunderLoan — Comprehensive Security Analysis Report

Audit Scope: ThunderLoan Protocol (flash loan lending)
Tools Used: Slither, Aderyn, Foundry (dynamic fuzz/exploit testing)
Contracts Analyzed: `ThunderLoan.sol`, `ThunderLoanUpgraded.sol`, `AssetToken.sol`, `OracleUpgradeable.sol`, Interfaces
Test Files: `ExploitTests.t.sol` (8 tests), `DeepSecurityProbeTest.sol` (8 tests)
Total Findings: 17 Unique Issues
Severity Classification: Immunefi Standard

Severity Classification (Immunefi Standard)

Severity	Count	Immunefi Criteria
Critical	5	Direct theft of user funds / Protocol insolvency
High	2	Theft of unclaimed yield / Temporary freezing of funds
Medium	4	Contract unable to operate / Griefing / Fails to deliver
Low	6	No value loss / Informational

CRITICAL

C-1: Reentrancy in `flashloan()` — Missing Reentrancy Guard

Severity: Critical — *Direct theft of any user funds, whether at-rest or in-motion*

DESCRIPTION

The `flashloan()` function in `ThunderLoan.sol` (lines 181–229) and `ThunderLoanUpgraded.sol` (lines 180–228) completely lacks a reentrancy guard modifier. The function records the `startingBalance` of the pool **before** making external calls to the untrusted receiver, then uses that **stale** `startingBalance` to verify repayment after the callback returns. During the callback (`executeOperation()`), the receiver can call **any** function on ThunderLoan or AssetToken because no reentrancy protection exists.

Execution flow in `flashloan()`: 1. Line 192: `startingBalance = IERC20(token).balanceOf(address(assetToken))` — snapshot 2. Line 204: `assetToken.updateExchangeRate(fee)` — external call 3. Line 208: `s_currentlyFlashLoaning[token] = true` — state write 4. Line 209: `assetToken.transferUnderlyingTo(receiverAddress, amount)` — tokens leave pool 5. Lines 211–222: `receiverAddress.functionCall(...)` — **untrusted callback** — NO REENTRANCY GUARD 6. Lines 224–225: `endingBalance < startingBalance + fee` — checks stale `startingBalance`

IMPACT

An attacker can deploy a malicious receiver contract that calls `redeem()` during the callback to drain the pool's underlying tokens, making the flash loan unrepayable and permanently stealing LP funds. Alternatively, calling `deposit()` instead of `repay()` mints AssetTokens at the current exchange rate, passing the balance check while diluting existing LPs.

PROOF OF CODE

This proof-of-code deploys a malicious flash loan receiver that calls `deposit()` instead of `repay()` during the execution callback. By depositing the borrowed funds plus the fee back into the pool, the receiver mints AssetTokens at the current exchange rate — converting the flash loan position into a permanent LP position while passing the ending balance check. This dilutes existing LPs and proves the missing reentrancy guard.

```
// test/mocks/DepositDuringFlashLoanReceiver.sol
contract DepositDuringFlashLoanReceiver {
    // Called by ThunderLoan.flashloan() during the untrusted callback
    function executeOperation(
        address token, uint256 amount, uint256 fee, address, bytes calldata
    ) external returns (bool) {
        uint256 total = amount + fee;
        IERC20(token).approve(msg.sender, total);
        // Instead of calling repay(), calls deposit() — mints AssetTokens
        // This passes the endingBalance &gt;= startingBalance + fee check
        // while the attacker keeps newly minted AssetTokens
        (bool success,) = msg.sender.call(
            abi.encodeWithSignature(&quot;deposit(address,uint256)&quot;, token, total)
        );
        require(success, &quot;deposit failed&quot;);
        return true;
    }
}
```

Test verification: - `testNoReentrancyGuardOnFlashLoan()` — Confirms re-entering `flashloan()` from inside the callback is allowed - `testDepositDuringFlashLoanInsteadOfRepay()` — Confirms `deposit()` mints AssetTokens, diluting LPs - `testRedeemDuringFlashLoanDrainsPool()` — Confirms `redeem()` is callable during active flash loan

---

RECOMMENDATION / MITIGATION

Add the `ReentrancyGuardUpgradeable` contract and apply the `nonReentrant` modifier to `flashloan()`. Also add a reentrancy check to `redeem()` to prevent pool draining during active flash loans.

```
<span class="gi">+ import { ReentrancyGuardUpgradeable } from</span>
<span class="gi">+     &quot;@openzeppelin/contracts-upgradeable/security/
ReentrancyGuardUpgradeable.sol&quot;;</span>

<span class="gd">- contract ThunderLoan is Initializable, OwnableUpgradeable, UUPSUpgradeable,
OracleUpgradeable {</span>
<span class="gi">+ contract ThunderLoan is Initializable, OwnableUpgradeable, UUPSUpgradeable,</
span>
<span class="gi">+     OracleUpgradeable, ReentrancyGuardUpgradeable {</span>

<span class="gi">+     // Add nonReentrant to flashloan()</span>
<span class="w">  </span>     function flashloan(
<span class="w">  </span>         address receiverAddress, IERC20 token, uint256 amount, bytes
calldata params
<span class="gd">-     ) external revertIfZero(amount) revertIfNotAllowedToken(token) {</span>
<span class="gi">+     ) external nonReentrant revertIfZero(amount) revertIfNotAllowedToken(token)
{</span>
<span class="w">  </span>         // ... existing logic stays the same ...
<span class="w">  </span>     }

<span class="gi">+     // Add reentrancy check to redeem()</span>
<span class="w">  </span>     function redeem(IERC20 token, uint256 amountOfAssetToken)
<span class="w">  </span>         external revertIfZero(amountOfAssetToken)
revertIfNotAllowedToken(token)
<span class="w">  </span>     {
<span class="gi">+         if (s_currentlyFlashLoaning[token]) {</span>
<span class="gi">+             revert ThunderLoan__NotCurrentlyFlashLoaning();</span>
<span class="gi">+         }</span>
<span class="w">  </span>         // ... rest of redeem logic stays the same ...
<span class="w">  </span>     }
<span class="w">  </span> }
```

CRITICAL

C-2: Exchange Rate Miscalculation Leading to Protocol Insolvency

Severity: Critical — *Protocol insolvency*

DESCRIPTION

The `AssetToken` exchange rate is computed via a purely mathematical formula that **always increases** regardless of the actual token balance in the pool, and is updated **before** tokens are transferred. This creates a permanent divergence between the stated virtual value (`totalSupply × exchangeRate`) and the actual underlying balance, making the protocol structurally insolvent from the very first deposit.

Root Cause 1 — Formula ignores actual balance:

```
// AssetToken.sol line 89
uint256 newExchangeRate = s_exchangeRate * (totalSupply() + fee) / totalSupply();
```

The rate is a monotonic accumulator that only goes up, no matter what the pool actually holds. During a flash loan, the pool balance drops to near-zero while the rate continues increasing.

Root Cause 2 — Rate updated before transfer:

```
// ThunderLoan.sol deposit() lines 153-156
assetToken.mint(msg.sender, mintAmount);
uint256 calculatedFee = getCalculatedFee(token, amount);
assetToken.updateExchangeRate(calculatedFee);  // &lt;-- Rate increased BEFORE transfer
token.safeTransferFrom(msg.sender, address(assetToken), amount); // &lt;-- transfer AFTER
```

IMPACT

After the very first deposit:

```
Deposit 100 tokens → mint 100 AssetTokens at rate 1.0
Then updateExchangeRate(0.3) → new rate = 1.003
Virtual value = 100 × 1.003 = 100.3
Actual balance = 100.0
```

The protocol owes **100.3** tokens but only has **100.0**. The last redeemer cannot withdraw their full value. This gap widens with every operation.

PROOF OF CODE

This test measures the virtual value (`totalSupply × exchangeRate / PRECISION`) and the actual underlying token balance immediately after the first deposit. It proves that the protocol is structurally insolvent from the very first transaction because the exchange rate was updated before the underlying tokens arrived via `safeTransferFrom`.

```
function testExchangeRateUpdatedBeforeTokensArrive() public {
    AssetToken assetToken = thunderLoan.getAssetFromToken(tokenA);
    uint256 underlying = tokenA.balanceOf(address(assetToken));
    uint256 supply = assetToken.totalSupply();
    uint256 rate = assetToken.getExchangeRate();
    uint256 virtualValue = (supply * rate) / assetToken.EXCHANGE_RATE_PRECISION();
    // Virtual value (100.3) exceeds actual underlying (100.0) from the first deposit
    assertTrue(virtualValue &gt; underlying,
        &quot;Protocol insolvent from first deposit&quot;);
}
```

Test verification: - `testExchangeRateUpdatedBeforeTokensArrive()` — Virtual > actual after first deposit - `testExchangeRateDivergesFromActualBalance()` — Gap widens with every operation - `testRoundingErrors()` — Dust and zero-fee rounding exacerbates the divergence

---

RECOMMENDATION / MITIGATION

Two changes are needed. First, fix `updateExchangeRate()` in `AssetToken.sol` to base the rate on the actual underlying balance rather than a mathematical accumulator. Second, reorder `deposit()` in `ThunderLoan.sol` to transfer tokens **before** updating the exchange rate.

```
<span class="w"> </span> // === AssetToken.sol: Fix updateExchangeRate to use actual balance ===
<span class="w"> </span> function updateExchangeRate(uint256 fee) external onlyThunderLoan {
<span class="gi">+     uint256 underlyingBalance = i_underlying.balanceOf(address(this));</span>
<span class="gi">+     uint256 newExchangeRate = (underlyingBalance * EXCHANGE_RATE_PRECISION()) /
totalSupply();</span>
<span class="gd">-     uint256 newExchangeRate = s_exchangeRate * (totalSupply() + fee) /
totalSupply();</span>
<span class="w"> </span>     if (newExchangeRate &lt;= s_exchangeRate) {
<span class="w"> </span>         revert AssetToken__ExhangeRateCanOnlyIncrease(s_exchangeRate,
newExchangeRate);
<span class="w"> </span>     }
<span class="w"> </span>     s_exchangeRate = newExchangeRate;
<span class="gi">+     emit ExchangeRateUpdated(s_exchangeRate);</span>
<span class="w"> </span> }
```

```
<span class="w"> </span> // === ThunderLoan.sol: Reorder deposit() — transfer BEFORE rate update
===
<span class="w"> </span> function deposit(IERC20 token, uint256 amount) external ... {
<span class="w"> </span>     AssetToken assetToken = s_tokenToAssetToken[token];
<span class="gi">+     // Transfer tokens FIRST — the rate update should reflect actual balance</
span>
<span class="gi">+     token.safeTransferFrom(msg.sender, address(assetToken), amount);</span>
<span class="gi">++</span>
<span class="w"> </span>     uint256 exchangeRate = assetToken.getExchangeRate();
<span class="w"> </span>     uint256 mintAmount = (amount *
assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
<span class="w"> </span>     emit Deposit(msg.sender, token, amount);
<span class="w"> </span>     assetToken.mint(msg.sender, mintAmount);
<span class="w"> </span>     uint256 calculatedFee = getCalculatedFee(token, amount);
<span class="w"> </span>     assetToken.updateExchangeRate(calculatedFee);
<span class="gd">-     token.safeTransferFrom(msg.sender, address(assetToken), amount);</span>
<span class="w"> </span> }
```

CRITICAL

C-3: Fee-on-Transfer Tokens Break Protocol Solvency

Severity: Critical — *Protocol insolvency*

DESCRIPTION

The protocol does not account for fee-on-transfer tokens that deduct a percentage on every transfer. When `deposit()` calls `safeTransferFrom(msg.sender, assetToken, amount)`, the actual amount received by the AssetToken contract is `amount - fee`, but all accounting (minting, exchange rate update) uses the full `amount` parameter. This creates an immediate insolvency.

IMPACT

With a 5% fee-on-transfer token: - User deposits 100 tokens → only 95 arrive - User receives 100 AssetTokens (minted based on 100) - Exchange rate updated based on a fee calculated on 100 - Virtual value (`supply × rate`) permanently exceeds actual pool balance - The last LP to redeem cannot withdraw their full value

PROOF OF CODE

This test uses a custom `WeirdERC20Mock` token that deducts a 5% fee on every transfer. After calling `deposit()` with this token, the code compares the virtual value (derived from the exchange rate and total supply) against the actual token balance in the AssetToken contract. The virtual value is higher, proving the protocol is insolvent with fee-on-transfer tokens.

```
// test/mocks/WeirdERC20Mock.sol — Token that takes 5% on every transfer
weirdToken.enableFeeOnTransfer(true);
weirdToken.setTransferFeePercent(5);

// After deposit, measure the accounting discrepancy:
uint256 underlying = weirdToken.balanceOf(address(assetToken));
uint256 virtualValue = (assetToken.totalSupply() * assetToken.getExchangeRate())
    / assetToken.EXCHANGE_RATE_PRECISION();
// Virtual value exceeds actual underlying — protocol cannot pay all LPs
assertTrue(virtualValue &gt; underlying, &quot;Insolvent from fee-on-transfer&quot;);
```

Test: `testFeeOnTransferTokenBreaksAccounting()` — **PASS**

RECOMMENDATION / MITIGATION

In `deposit()`, measure the actual balance change after `safeTransferFrom` instead of relying on the `amount` parameter. This correctly accounts for any transfer fees deducted by the token.

```
 function deposit(IERC20 token, uint256 amount) external ... {
     AssetToken assetToken = s_tokenToAssetToken[token];

+    // Measure actual balance change after transfer
+    uint256 balanceBefore = token.balanceOf(address(assetToken));
     token.safeTransferFrom(msg.sender, address(assetToken), amount);
+    uint256 actualAmount = token.balanceOf(address(assetToken)) -
balanceBefore;

-    uint256 exchangeRate = assetToken.getExchangeRate();
-    uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) /
exchangeRate;
-    emit Deposit(msg.sender, token, amount);
-    assetToken.mint(msg.sender, mintAmount);
+    uint256 exchangeRate = assetToken.getExchangeRate();
+    uint256 mintAmount = (actualAmount *
assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
+    emit Deposit(msg.sender, token, actualAmount);
+    assetToken.mint(msg.sender, mintAmount);

-    uint256 calculatedFee = getCalculatedFee(token, amount);
+    uint256 calculatedFee = getCalculatedFee(token, actualAmount);
     assetToken.updateExchangeRate(calculatedFee);
-    token.safeTransferFrom(msg.sender, address(assetToken), amount);
 }
```

CRITICAL

C-4: Centralization — Single Owner With No Timelock

Severity: Critical — *Direct theft of any user funds, whether at-rest or in-motion*

DESCRIPTION

The contract uses `OwnableUpgradeable` with a **single owner** and **zero timelocks** on any owner function. The owner can perform the following actions instantly in a single transaction:

1. **Change fees** — `updateFlashLoanFee()` — from 0.3% to 50% in one block
2. **Remove tokens** — `setAllowedToken(token, false)` — trapping all LP AssetTokens
3. **Upgrade implementation** — `_authorizeUpgrade()` — deploy malicious bytecode
4. **Transfer ownership** — `transferOwnership()` — no multi-sig required

IMPACT

A compromised or malicious owner key = **total protocol theft**. The owner can deploy a malicious upgraded implementation that drains all LP funds, remove all supported tokens trapping LP deposits permanently, set the flash loan fee to 100% stealing all borrower value, or transfer ownership to any address in one transaction.

PROOF OF CODE

This test demonstrates two critical capabilities. First, the fee can be changed from 0.3% to 50% in a single transaction with zero delay — there is no governance vote, no timelock, no multi-sig check. Second, ownership can be transferred to any arbitrary address in one call — no confirmation, no cooldown.

```
// PoC 1: Change fee from 0.3% to 50% instantly — no timelock
uint256 oldFee = thunderLoan.getFee(); // 3e15 = 0.3%
vm.prank(owner);
thunderLoan.updateFlashLoanFee(5e17); // 50% — one transaction, zero delay
assertFalse(thunderLoan.getFee() == oldFee, &amp;quot;Fee changed instantly&amp;quot;);

// PoC 2: Transfer ownership to any address — no multi-sig
address malicious = makeAddr(&amp;quot;malicious&amp;quot;);
vm.prank(owner);
thunderLoan.transferOwnership(malicious);
assertEq(thunderLoan.owner(), malicious, &amp;quot;Owner transferred in 1 tx&amp;quot;);
```

Test verification: - `testNoTimelockOnOwnerActions()` — All owner functions have zero delay - `testSingleOwnerGovernance()` — Ownership can be transferred to any address - `testFrontrunningFeeChange()` — MEV bots profit from instant fee changes

RECOMMENDATION / MITIGATION

Add a commit-reveal timelock pattern to all sensitive owner functions. This gives users a window to react before changes take effect. Additionally, use a multi-sig wallet as the contract owner.

```
<span class="gi">+ // Add timelock state variables</span>
<span class="gi">+ uint256 public constant OWNER_DELAY = 2 days;</span>
<span class="gi">+ mapping(bytes32 =&gt; uint256) public s_pendingActions;</span>
<span class="gi">+ // Track token removal for withdrawal grace period</span>
<span class="gi">+ mapping(IERC20 =&gt; uint256) public s_tokenRemovalTime;</span>
<span class="gi">+ mapping(IERC20 =&gt; bool) public s_withdrawalOnly;</span>

<span class="w"> </span> // Replace instantaneous setAllowedToken with scheduled + execute pattern
<span class="gi">+ function scheduleSetAllowedToken(IERC20 token, bool allowed) external onlyOwner
{</span>
<span class="gi">+     bytes32 actionId = keccak256(abi.encodePacked(&quot;setAllowedToken&quot;,
token, allowed));</span>
<span class="gi">+     s_pendingActions[actionId] = block.timestamp + OWNER_DELAY;</span>
<span class="gi">+     emit ActionScheduled(actionId, block.timestamp + OWNER_DELAY);</span>
<span class="gi">+ }</span>

<span class="gi">+ function executeSetAllowedToken(IERC20 token, bool allowed) external onlyOwner
{</span>
<span class="gi">+     bytes32 actionId = keccak256(abi.encodePacked(&quot;setAllowedToken&quot;,
token, allowed));</span>
<span class="gi">+     require(s_pendingActions[actionId] != 0, &quot;Action not
scheduled&quot;);</span>
<span class="gi">+     require(s_pendingActions[actionId] &lt;= block.timestamp, &quot;Timelock
not expired&quot;);</span>
<span class="gi">+     delete s_pendingActions[actionId];</span>
<span class="gi">+     _setAllowedToken(token, allowed);</span>
<span class="gi">+ }</span>

<span class="w"> </span> // In _setAllowedToken, add withdrawal grace period when removing a
token:
<span class="gi">+ function _setAllowedToken(IERC20 token, bool allowed) internal {</span>
<span class="gi">+     if (!allowed) {</span>
<span class="gi">+         s_withdrawalOnly[token] = true;</span>
<span class="gi">+         s_tokenRemovalTime[token] = block.timestamp + 7 days;</span>
<span class="gi">+         // Allow LPs to withdraw during the grace period</span>
<span class="gi">+     }</span>
<span class="gi">+     // ... existing logic ...</span>
<span class="gi">+ }</span>

<span class="w"> </span> // In redeem(), allow withdrawal-only mode:
<span class="w"> </span> function redeem(IERC20 token, uint256 amountOfAssetToken) external ... {
<span class="gi">+     if (!isAllowedToken(token) &amp;&amp; !s_withdrawalOnly[token]) {</span>
<span class="gi">+         revert ThunderLoan__NotAllowedToken(token);</span>
<span class="gi">+     }</span>
<span class="gi">+     if (s_withdrawalOnly[token] &amp;&amp; s_tokenRemovalTime[token] &lt;
block.timestamp) {</span>
<span class="gi">+         revert ThunderLoan__WithdrawalPeriodExpired();</span>
<span class="gi">+     }</span>
<span class="w"> </span>     // ... existing logic ...
<span class="w"> </span> }
```

CRITICAL

C-5: Storage Collision on UUPS Upgrade

Severity: Critical — *Protocol insolvency*

DESCRIPTION

`ThunderLoanUpgraded.sol` **removed** `s_feePrecision` from the state variable declarations, shifting all subsequent storage slots by one position. The upgraded implementation's `getFee()` reads from slot N+1, which was `s_feePrecision` (value: `1e18` = 100%) in the original contract — NOT `s_flashLoanFee` (value: `3e15` = 0.3%).

STORAGE LAYOUT CORRUPTION

Slot	ThunderLoan (original)	ThunderLoanUpgraded
N	`s_tokenToAssetToken`	`s_tokenToAssetToken`
N+1	**`s_feePrecision`** (1e18)	`s_flashLoanFee` ← reads 1e18!
N+2	`s_flashLoanFee` (3e15)	`s_currentlyFlashLoaning` ← corrupted
N+3	`s_currentlyFlashLoaning`	`s_poolFactory` ← corrupted
N+4	`s_poolFactory` (oracle)	—

IMPACT

After upgrading from `ThunderLoan` to `ThunderLoanUpgraded`: - `getFee()` returns `1e18` (100%) instead of `3e15` (0.3%) - Every flash loan costs 100% of the borrowed amount - `s_currentlyFlashLoaning` and `s_poolFactory` read from wrong slots, corrupting flash loan tracking and oracle price lookups

PROOF OF CODE

This test deploys an original `ThunderLoan` proxy, upgrades it to `ThunderLoanUpgraded`, and reads `getFee()`. The value returned is `1e18` (the old `s_feePrecision` value) instead of `3e15` (the actual flash loan fee), proving the storage slot mismatch.

```
// Deploy original and upgrade to ThunderLoanUpgraded
ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
vm.prank(owner);
thunderLoan3.upgradeToAndCall(address(upgraded), &amp;quot;&amp;quot;);

ThunderLoanUpgraded proxy = ThunderLoanUpgraded(address(proxy3));
uint256 upgradedFee = proxy.getFee();

// getFee() reads from s_feePrecision slot (1e18) instead of s_flashLoanFee (3e15)
assertEq(upgradedFee, 1e18, &amp;quot;Storage collision: reads wrong slot&amp;quot;);
assertFalse(upgradedFee == 3e15, &amp;quot;Should NOT return the original fee&amp;quot;);
```

Test: `testUninitializedProxy()` — **PASS**

RECOMMENDATION / MITIGATION

Never remove state variables in upgradeable contracts — the storage layout must remain identical across all versions. Restore `s_feePrecision` as a deprecated storage slot and add storage gaps in base contracts for future-proofing.

```
<span class="w"> </span> // === ThunderLoanUpgraded.sol: Restore storage compatibility ===
<span class="w"> </span> contract ThunderLoanUpgraded is Initializable, OwnableUpgradeable,
UUPSUpgradeable, OracleUpgradeable {
<span class="w"> </span>     // ... error declarations stay the same ...

<span class="w"> </span>     mapping(IERC20 =&gt; AssetToken) public s_tokenToAssetToken;

<span class="gd">-     // REMOVED: uint256 private s_feePrecision;</span>
<span class="gi">+     // Deprecated — kept for storage layout compatibility (was
s_feePrecision)</span>
<span class="gi">+     uint256 private s_deprecated_feePrecision;</span>

<span class="gi">+     // s_flashLoanFee stays at the same storage slot as before</span>
<span class="w"> </span>     uint256 private s_flashLoanFee;
<span class="gd">-     uint256 public constant FEE_PRECISION = 1e18;</span>
<span class="gi">+     // Use constant instead of storage now:</span>
<span class="gi">+     uint256 public constant FEE_PRECISION = 1e18;</span>

<span class="w"> </span>     // ... rest of contract stays the same ...
<span class="w"> </span> }
```

```
<span class="w"> </span> // === OracleUpgradeable.sol: Add storage gaps ===
<span class="w"> </span> contract OracleUpgradeable is Initializable {
<span class="w"> </span>     address private s_poolFactory;
<span class="gi">+     // Storage gap — reserve slots for future upgrades without shifting</span>
<span class="gi">+     uint256[20] private __gap;</span>
<span class="w"> </span>     // ...
<span class="w"> </span> }
```

HIGH

H-1: Oracle Price Manipulation — Theft of LP Yield

Severity: High — *Theft of unclaimed yield*

DESCRIPTION

`getCalculatedFee()` computes the flash loan fee by querying a **spot price** from an external AMM pool (T-Swap). The function `getPriceInWeth(address(token))` retrieves the current price from the pool, which can be manipulated via large swaps in the same transaction. An attacker can manipulate the pool's price ratio before taking a flash loan, reducing fees to near-zero and stealing the yield that should accrue to LPs.

```
// ThunderLoan.sol line 258-263
function getCalculatedFee(IERC20 token, uint256 amount) public view returns (uint256 fee) {
    uint256 valueOfBorrowedToken = (amount * getPriceInWeth(address(token))) / s_feePrecision;
    fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;
}

// OracleUpgradeable.sol line 19-22
function getPriceInWeth(address token) public view returns (uint256) {
    address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(token);
    return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth();
}
```

IMPACT

- Fee reduced by **55%** in testing by swapping 500e18 pool tokens for WETH. With larger swaps, fees approach zero.
- Combined with the reentrancy vulnerability (C-1), the oracle price can be manipulated **during** an active flash loan callback.
- LPs receive less fee yield than expected, effectively stolen by the attacker.

PROOF OF CODE

This test deploys a real `BuffMockTSwap` AMM pool and adds deep liquidity. It measures the flash loan fee before and after manipulating the pool price by selling 500e18 pool tokens for WETH, which drains WETH reserves and changes the token price. The fee is significantly lower after manipulation, proving the oracle-dependent fee is manipulable.

```
// Manipulate AMM pool price by selling pool tokens for WETH
// This drains WETH reserves, changing the price-of-one-pool-token-in-WETH
vm.startPrank(attacker);
pool.swapPoolTokenForWethBasedOnInputPoolToken(500e18, 1, block.timestamp + 1 hours);
vm.stopPrank();

uint256 manipulatedFee = thunderLoan2.getCalculatedFee(tokenA, FLASH_LOAN_AMOUNT);
// Fee is significantly lower than the pre-manipulation fee
assertTrue(manipulatedFee &lt; normalFee, &quot;Oracle manipulation reduces fee&quot;);
```

Test verification: - `testOracleManipulationReducesFee()` — **PASS** — Fee reduced by 55% - `testOracleManipulationDuringCallback()` — **PASS** — Price manipulated during callback

---

RECOMMENDATION / MITIGATION

Replace the spot price query with a time-weighted average price (TWAP) to resist flash loan-based manipulation. Also add a minimum fee floor to prevent fees from rounding to zero.

```
<span class="w"> </span> // === OracleUpgradeable.sol: Replace spot price with TWAP ===
<span class="w"> </span> function getPriceInWeth(address token) public view returns (uint256) {
<span class="w"> </span>     address pool = IPoolFactory(s_poolFactory).getPool(token);
<span class="gd">-     return ITSwapPool(pool).getPriceOfOnePoolTokenInWeth();</span>
<span class="gi">+     // Use time-weighted average price over the last 30 minutes</span>
<span class="gi">+     // This resists flash loan-based manipulation of the spot price</span>
<span class="gi">+     return ITwapOracle(pool).getTwap(1800);</span>
<span class="w"> </span> }
```

```
<span class="w"> </span> // === ThunderLoan.sol: Add minimum fee floor ===
<span class="w"> </span> function getCalculatedFee(IERC20 token, uint256 amount) public view
returns (uint256 fee) {
<span class="w"> </span>     uint256 valueOfBorrowedToken = (amount *
getPriceInWeth(address(token))) / s_feePrecision;
<span class="w"> </span>     fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;
<span class="gi">+     // Minimum fee floor — prevents rounding to zero and oracle manipulation</
span>
<span class="gi">+     uint256 minFee = 1e15; // 0.001 token worth</span>
<span class="gi">+     if (fee &amp;lt; minFee) fee = minFee;</span>
<span class="w"> </span> }
```

HIGH

H-2: Token Removal Traps LP Funds Permanently

Severity: High — *Temporary freezing of funds*

DESCRIPTION

When the owner calls `setAllowedToken(token, false)`, the `s_tokenToAssetToken[token]` mapping is deleted. LPs holding AssetTokens for that token can no longer call `redeem()` — the function reverts with `ThunderLoan__NotAllowedToken` because the token is no longer in the allowed mapping. The AssetToken contract still holds the underlying tokens, but LPs have no path to withdraw.

```
// ThunderLoan.sol line 251 — the destructive line
delete s_tokenToAssetToken[token]; // Mapping deleted instantly — LP access revoked
```

IMPACT

- LP AssetTokens become permanently stuck after token removal
- No withdrawal grace period exists
- The AssetToken contract still exists and holds the underlying tokens, but LPs cannot access them through the protocol
- The owner could re-allow the token to restore access, but there is no guarantee they will

PROOF OF CODE

This test allows the owner to remove a token, then attempts to call `redeem()` as an LP. The call reverts with `ThunderLoan__NotAllowedToken` even though the LP still holds their AssetTokens and the underlying funds are still in the AssetToken contract. The LP's funds are permanently trapped.

```
// LP holds AssetTokens before removal
AssetToken assetToken = thunderLoan.getAssetFromToken(tokenA);
uint256 stuckBalance = assetToken.balanceOf(liquidityProvider); // e.g., 5e19

// Owner removes the token — mapping is deleted instantly
thunderLoan.setAllowedToken(tokenA, false);

// LP tries to redeem — reverts because token is no longer allowed
vm.startPrank(liquidityProvider);
vm.expectRevert(abi.encodeWithSelector(
    ThunderLoan.ThunderLoan__NotAllowedToken.selector, tokenA
));
thunderLoan.redeem(tokenA, stuckBalance); // &lt;-- ALWAYS reverts, funds stuck
vm.stopPrank();
```

Test: `testAccessControlTokenRemovalTrapsFunds()` — **PASS**

RECOMMENDATION / MITIGATION

Add a withdrawal-only grace period when a token is removed. Instead of instantly deleting the mapping, switch the token to "withdrawal-only" mode for a fixed period, allowing existing LPs to redeem their positions.

```
<span class="gi">+ // New state variables</span>
<span class="gi">+ mapping(IERC20 =&gt; bool) public s_withdrawalOnly;</span>
<span class="gi">+ mapping(IERC20 =&gt; uint256) public s_tokenRemovalTime;</span>

<span class="w"> </span> // Modify setAllowedToken to use withdrawal-only mode instead of instant
deletion
<span class="gi">+ function scheduleTokenRemoval(IERC20 token) external onlyOwner {</span>
<span class="gi">+     emit TokenRemovalScheduled(token, block.timestamp + 7 days);</span>
<span class="gi">+ }</span>

<span class="w"> </span> function setAllowedToken(IERC20 token, bool allowed) external onlyOwner
returns (AssetToken) {
<span class="w"> </span>     if (!allowed) {
<span class="gd">-         delete s_tokenToAssetToken[token];</span>
<span class="gi">+         // Switch to withdrawal-only mode instead of deleting</span>
<span class="gi">+         s_withdrawalOnly[token] = true;</span>
<span class="gi">+         s_tokenRemovalTime[token] = block.timestamp + 7 days;</span>
<span class="w"> </span>         emit AllowedTokenSet(token, s_tokenToAssetToken[token], allowed);
<span class="gd">-         return s_tokenToAssetToken[token];</span>
<span class="gi">+         return s_tokenToAssetToken[token]; // AssetToken still accessible</
span>
<span class="w"> </span>     }
<span class="w"> </span>     // ... existing logic for adding tokens ...
<span class="w"> </span> }

<span class="w"> </span> // Update redeem() to allow withdrawals during the grace period
<span class="w"> </span> function redeem(IERC20 token, uint256 amountOfAssetToken) external ... {
<span class="gd">-     if (!isAllowedToken(token)) revert ThunderLoan__NotAllowedToken(token);</
span>
<span class="gi">+     if (!isAllowedToken(token) &amp;&amp; !s_withdrawalOnly[token]) {</span>
<span class="gi">+         revert ThunderLoan__NotAllowedToken(token);</span>
<span class="gi">+     }</span>
<span class="gi">+     if (s_withdrawalOnly[token] &amp;&amp; block.timestamp &gt;
s_tokenRemovalTime[token]) {</span>
<span class="gi">+         revert ThunderLoan__WithdrawalPeriodExpired();</span>
<span class="gi">+     }</span>
<span class="w"> </span>     // ... existing redeem logic ...
<span class="w"> </span> }
```

MEDIUM

M-1: Missing Zero-Address Checks Across All Contracts

Severity: Medium — *Smart contract unable to operate due to lack of token funds (Immunefi)*

DESCRIPTION

Several functions accept `address` parameters without validating that they are not `address(0)`. If a zero address is passed for a critical parameter such as the pool factory, the oracle, the `tswapAddress` initializer, or token receivers, the contract can become permanently broken or funds can be locked.

Vulnerable locations flagged during manual audit:

`OracleUpgradeable.sol` — every address parameter should be checked for zero:

```
// @audit every address param should be checked if zero address.
// Create a revertIfZeroAddress modifier.
function __Oracle_init(address poolFactoryAddress) internal onlyInitializing {
    __Oracle_init_unchained(poolFactoryAddress);
}

function __Oracle_init_unchained(address poolFactoryAddress) internal onlyInitializing {
    s_poolFactory = poolFactoryAddress; // &lt;-- No zero-address check
}

function getPriceInWeth(address token) public view returns (uint256) {
    address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(token);
    return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth();
}

function getPrice(address token) external view returns (uint256) {
    return getPriceInWeth(token);
}
```

`ThunderLoan.sol` — `initialize()` passes `tswapAddress` unchecked into the oracle:

```
// @audit tswapAddress param should be checked if zeroAddress
function initialize(address tswapAddress) external initializer {
    __Ownable_init(msg.sender);
    __UUPSUpgradeable_init();
    __Oracle_init(tswapAddress); // &lt;-- No zero-address check on tswapAddress
    s_feePrecision = 1e18;
    s_flashLoanFee = 3e15; // 0.3% ETH fee
}
```

`ThunderLoanUpgraded.sol` — same issue in the upgraded contract:

```
function initialize(address tswapAddress) external initializer {
    __Ownable_init(msg.sender);
    __UUPSUpgradeable_init();
    __Oracle_init(tswapAddress); // &lt;-- Same missing zero-address check
    s_flashLoanFee = 3e15; // 0.3% ETH fee
}
```

SUMMARY OF UNCHECKED ADDRESS PARAMETERS

Contract	Function	Parameter	Consequence if `address(0)`
`OracleUpgradeable.sol`	`__Oracle_init`	`poolFactoryAddress`	Oracle broken — all price calls revert
`OracleUpgradeable.sol`	`__Oracle_init_unchained`	`poolFactoryAddress`	Oracle broken — all price calls revert
`OracleUpgradeable.sol`	`getPriceInWeth`	`token`	Reverts on zero-token lookup
`OracleUpgradeable.sol`	`getPrice`	`token`	Reverts on zero-token lookup
`ThunderLoan.sol`	`initialize`	`tswapAddress`	**Contract bricked after deployment**
`ThunderLoanUpgraded.sol`	`initialize`	`tswapAddress`	**Contract bricked after deployment**
`ThunderLoan.sol`	`repay`	`token`	No `revertIfNotAllowedToken` guard
`ThunderLoanUpgraded.sol`	`repay`	`token`	No `revertIfNotAllowedToken` guard
`AssetToken.sol`	`mint`	`to`	AssetTokens minted to `address(0)` — permanently burnt
`AssetToken.sol`	`transferUnderlyingTo`	`to`	Underlying tokens sent to `address(0)` — permanently burnt

---

IMPACT

If `initialize()` is called with `tswapAddress == address(0)` (or the proxy is initialized with a zero pool factory), the protocol is **permanently bricked**: - `getPriceInWeth()` calls `IPoolFactory(address(0)).getPool(token)` → reverts - Every `deposit()`, `flashloan()`, and `getCalculatedFee()` call reverts - The contract cannot be re-initialized because `initializer` runs only once - `mint(0, ...)` and `transferUnderlyingTo(0, ...)` would permanently burn user funds

---

PROOF OF CODE

This is a static code issue — the vulnerable functions accept any address without validation. If `address(0)` is passed for `tswapAddress` during initialization, the contract deploys successfully but every price-dependent function reverts afterward, making the protocol permanently inoperable.

```
// If the deployer accidentally (or maliciously) initializes with address(0):
function __Oracle_init_unchained(address poolFactoryAddress) internal onlyInitializing {
    s_poolFactory = poolFactoryAddress; // No revert if address(0) is passed
}

// Later, any call to getCalculatedFee() leads to:
//   IPoolFactory(address(0)).getPool(token)  →  REVERTS every time
// deposit(), flashloan(), getFee() → all permanently broken
```

RECOMMENDATION / MITIGATION

Create a reusable `revertIfZeroAddress` modifier and apply it to **every** function that accepts an address or `IERC20` parameter. Also add a zero-address check in `initialize()` for `tswapAddress`.

```
<span class="gi">+ // Add the modifier and error in a shared/base location (e.g.,
OracleUpgradeable)</span>
<span class="gi">+ error ThunderLoan__ZeroAddress();</span>

<span class="gi">+ modifier revertIfZeroAddress(address someAddress) {</span>
<span class="gi">+     if (someAddress == address(0)) {</span>
<span class="gi">+         revert ThunderLoan__ZeroAddress();</span>
<span class="gi">+     }</span>
<span class="gi">+     _;</span>
<span class="gi">+ }</span>

<span class="w"> </span> // === OracleUpgradeable.sol ===
<span class="gd">- function __Oracle_init(address poolFactoryAddress) internal onlyInitializing
{</span>
<span class="gi">+ function __Oracle_init(address poolFactoryAddress)</span>
<span class="gi">+     internal onlyInitializing revertIfZeroAddress(poolFactoryAddress)</span>
<span class="gi">+ {</span>
<span class="w"> </span>     __Oracle_init_unchained(poolFactoryAddress);
<span class="w"> </span> }

<span class="gd">- function __Oracle_init_unchained(address poolFactoryAddress) internal
onlyInitializing {</span>
<span class="gi">+ function __Oracle_init_unchained(address poolFactoryAddress)</span>
<span class="gi">+     internal onlyInitializing revertIfZeroAddress(poolFactoryAddress)</span>
<span class="gi">+ {</span>
<span class="w"> </span>     s_poolFactory = poolFactoryAddress;
<span class="w"> </span> }

<span class="gd">- function getPriceInWeth(address token) public view returns (uint256) {</span>
<span class="gi">+ function getPriceInWeth(address token) public view revertIfZeroAddress(token)
returns (uint256) {</span>
<span class="w"> </span>     address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(token);
<span class="w"> </span>     return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth();
<span class="w"> </span> }

<span class="gd">- function getPrice(address token) external view returns (uint256) {</span>
<span class="gi">+ function getPrice(address token) external view revertIfZeroAddress(token)
returns (uint256) {</span>
<span class="w"> </span>     return getPriceInWeth(token);
<span class="w"> </span> }

<span class="w"> </span> // === ThunderLoan.sol / ThunderLoanUpgraded.sol ===
<span class="gd">- function initialize(address tswapAddress) external initializer {</span>
<span class="gi">+ function initialize(address tswapAddress) external initializer
revertIfZeroAddress(tswapAddress) {</span>
<span class="w"> </span>     __Ownable_init(msg.sender);
<span class="w"> </span>     __UUPSUpgradeable_init();
<span class="w"> </span>     // @audit-fixed: tswapAddress now checked before being passed to the
oracle
<span class="w"> </span>     __Oracle_init(tswapAddress);
<span class="w"> </span>     s_feePrecision = 1e18;
<span class="w"> </span>     s_flashLoanFee = 3e15;
<span class="w"> </span> }

<span class="w"> </span> // === AssetToken.sol — prevent burning to address(0) ===
<span class="gd">- function mint(address to, uint256 amount) external onlyThunderLoan {</span>
<span class="gi">+ function mint(address to, uint256 amount)</span>
<span class="gi">+     external onlyThunderLoan revertIfZeroAddress(to)</span>
<span class="gi">+ {</span>
<span class="w"> </span>     _mint(to, amount);
<span class="w"> </span> }

<span class="gd">- function transferUnderlyingTo(address to, uint256 amount) external
onlyThunderLoan {</span>
<span class="gi">+ function transferUnderlyingTo(address to, uint256 amount)</span>
<span class="gi">+     external onlyThunderLoan revertIfZeroAddress(to)</span>
<span class="gi">+ {</span>
<span class="w"> </span>     i_underlying.safeTransfer(to, amount);
```

```
<span class="w"> </span> }

<span class="w"> </span> // === ThunderLoan.sol — validate token in repay() ===
<span class="gd">- function repay(IERC20 token, uint256 amount) public {</span>
<span class="gi">+ function repay(IERC20 token, uint256 amount)</span>
<span class="gi">+     public revertIfZeroAddress(address(token))</span>
<span class="gi">+ {</span>
<span class="w"> </span>     if (!s_currentlyFlashLoaning[token]) {
<span class="w"> </span>         revert ThunderLoan__NotCurrentlyFlashLoaning();
<span class="w"> </span>     }
<span class="w"> </span>     // ...
<span class="w"> </span> }
```

MEDIUM

M-2: Direct Donation Creates Accounting Discrepancy

Severity: Medium — *Griefing*

DESCRIPTION

Direct token transfers to the AssetToken contract (bypassing `deposit()`) increase the underlying token balance without updating the exchange rate or total supply. This means the actual pool balance no longer matches the virtual value derived from `totalSupply × exchangeRate / PRECISION`.

IMPACT

If a donor sends tokens directly, LPs can redeem for more than their stated value (the pool has more than expected). This discrepancy distorts LP value calculations and can be used in combination with other vulnerabilities for complex attacks.

PROOF OF CODE

This test sends tokens directly to the AssetToken contract address (bypassing `deposit()`) and then checks that the exchange rate is unchanged while the pool balance has increased. This creates a mismatch between what the exchange rate says LPs own and what the pool actually holds.

```
// Send 50 tokens directly to AssetToken contract — bypasses deposit()
tokenA.mint(attacker, 50e18);
vm.prank(attacker);
tokenA.transfer(address(assetToken), 50e18); // Direct transfer — no deposit()

// Exchange rate and total supply are unchanged by the donation
uint256 underlyingAfter = tokenA.balanceOf(address(assetToken));
uint256 rateAfter = assetToken.getExchangeRate();
assertEq(rateBefore, rateAfter, &amp;quot;Donation should NOT change exchange rate&amp;quot;);

// But the pool now has more underlying than virtual value suggests
uint256 virtualValue = (supplyBefore * rateAfter) / EXCHANGE_RATE_PRECISION;
assertTrue(underlyingAfter &amp;gt; virtualValue, &amp;quot;Pool balance exceeds virtual value&amp;quot;);
```

Test: `testDirectDonationInflatesBacking()` — **PASS**

RECOMMENDATION / MITIGATION

Track the expected total underlying balance alongside the exchange rate, and add a `skim()` function to handle accidental direct transfers.

```
<span class="gi">+ // Track total underlying across all deposits</span>
<span class="gi">+ uint256 public s_totalUnderlying;</span>

<span class="w"> </span> function deposit(IERC20 token, uint256 amount) external ... {
<span class="w"> </span>     // ...
<span class="gi">+     // Use balance-delta accounting to track actual amount received</span>
<span class="gi">+     uint256 balanceBefore = token.balanceOf(address(assetToken));</span>
<span class="w"> </span>     token.safeTransferFrom(msg.sender, address(assetToken), amount);
<span class="gi">+     uint256 actualAmount = token.balanceOf(address(assetToken)) -
balanceBefore;</span>
<span class="gi">+     s_totalUnderlying += actualAmount;</span>
<span class="w"> </span>     // ... rest of function ...
<span class="w"> </span> }

<span class="gi">+ // Allow recovery of accidentally transferred tokens</span>
<span class="gi">+ function skim(IERC20 token) external onlyOwner {</span>
<span class="gi">+     AssetToken assetToken = s_tokenToAssetToken[token];</span>
<span class="gi">+     uint256 actual = token.balanceOf(address(assetToken));</span>
<span class="gi">+     if (actual &gt; s_totalUnderlying) {</span>
<span class="gi">+         uint256 excess = actual - s_totalUnderlying;</span>
<span class="gi">+         token.safeTransfer(owner(), excess);</span>
<span class="gi">+     }</span>
<span class="gi">+ }</span>
```

MEDIUM

M-3: Frontrunning Fee Changes (MEV Opportunity)

Severity: Medium — *Griefing / Theft of unclaimed yield*

DESCRIPTION

Since `updateFlashLoanFee()` has no timelock or commit-reveal mechanism, MEV bots can observe pending fee change transactions in the public mempool and frontrun them. An attacker takes a flash loan at the old (lower) fee before a fee increase is mined, or withdraws liquidity before a fee decrease.

IMPACT

The attacker saves the difference between the old and new fee. In testing, an attacker borrowed at the old fee (0.3%) before a 50% fee increase — saving 2.485e19 wei on a 50e18 loan.

PROOF OF CODE

This test simulates an attacker who sees a pending `updateFlashLoanFee(50%)` transaction in the mempool. The attacker frontruns it by taking a flash loan at the current 0.3% fee. After the fee change is mined, the same loan would cost 50% — the attacker has saved the difference.

```
// Attacker sees pending fee increase in the mempool — frontruns with a flash loan
uint256 fee = thunderLoan.getCalculatedFee(tokenA, FLASH_LOAN_AMOUNT); // 0.3% fee
vm.prank(attacker);
thunderLoan.flashloan(address(receiver), tokenA, FLASH_LOAN_AMOUNT, &amp;quot;&amp;quot;); // Borrows at
0.3%

// Owner&amp;#39;s fee change tx goes through
vm.prank(owner);
thunderLoan.updateFlashLoanFee(5e17); // 50%

// The same loan would now cost 50% — attacker saved the difference
uint256 newFee = thunderLoan.getCalculatedFee(tokenA, FLASH_LOAN_AMOUNT);
assertTrue(newFee &amp;gt; fee, &amp;quot;Attacker saved by frontrunning&amp;quot;);
```

Test: `testFrontrunningFeeChange()` — **PASS**

RECOMMENDATION / MITIGATION

Split `updateFlashLoanFee()` into a two-step commit-reveal pattern with a timelock. This prevents MEV bots from frontrunning by giving users a window to observe proposed changes.

```
<span class="gi">+ // New state variables for timelocked fee changes</span>
<span class="gi">+ uint256 public s_pendingFee;</span>
<span class="gi">+ uint256 public s_pendingFeeDeadline;</span>

<span class="gd">- function updateFlashLoanFee(uint256 newFee) external onlyOwner {</span>
<span class="gi">+ function proposeFeeUpdate(uint256 newFee) external onlyOwner {</span>
<span class="w"> </span>     if (newFee &gt; s_feePrecision) revert ThunderLoan__BadNewFee();
<span class="gd">-     s_flashLoanFee = newFee;</span>
<span class="gi">+     s_pendingFee = newFee;</span>
<span class="gi">+     s_pendingFeeDeadline = block.timestamp + 2 days;</span>
<span class="gi">+     emit FeeUpdateProposed(newFee, s_pendingFeeDeadline);</span>
<span class="w"> </span> }

<span class="gi">+ function executeFeeUpdate() external onlyOwner {</span>
<span class="gi">+     require(s_pendingFeeDeadline != 0, &quot;No pending fee update&quot;);</
span>
<span class="gi">+     require(block.timestamp &gt;= s_pendingFeeDeadline, &quot;Timelock not
expired&quot;);</span>
<span class="gi">+     emit FlashLoanFeeUpdated(s_flashLoanFee, s_pendingFee);</span>
<span class="gi">+     s_flashLoanFee = s_pendingFee;</span>
<span class="gi">+     s_pendingFee = 0;</span>
<span class="gi">+     s_pendingFeeDeadline = 0;</span>
<span class="gi">+ }</span>
```

MEDIUM

M-4: Rounding Errors Cause Dust Loss and Zero-Fee Loans

Severity: Medium — *Contract fails to deliver promised returns*

DESCRIPTION

Multiple Solidity integer division operations truncate (floor) results, causing value loss for small amounts.

1. Deposit minting rounds to zero:

```
mintAmount = (amount * EXCHANGE_RATE_PRECISION) / exchangeRate;
```

When `amount * 1e18 < exchangeRate`, `mintAmount = 0`. The user transfers tokens but receives zero AssetTokens.

2. Fee calculation rounds to zero: `getCalculatedFee()` has two sequential divisions. For tiny amounts, both divisions round down, making flash loans free.

3. Redeem truncation:

```
amountUnderlying = (assetTokens * exchangeRate) / EXCHANGE_RATE_PRECISION;
```

Every redemption loses dust due to floor division, accumulating over time.

IMPACT

Users can lose their deposit of tiny amounts permanently. Flash loans for tiny amounts (`1 wei`) have zero fee. Accumulated dust from repeated redemptions stays in the protocol. The `updateExchangeRate(0)` call (when fee rounds to 0) reverts because the rate cannot increase, bricking tiny deposits.

PROOF OF CODE

This test inflates the exchange rate via a flash loan, then attempts to deposit 1 wei. The mint amount rounds to 0 and the fee rounds to 0. The deposit reverts because `updateExchangeRate(0)` cannot increase the rate — proving that small depositors lose their funds permanently.

```
// After enough deposits to inflate exchangeRate &gt; 1e18:
uint256 tinyAmount = 1; // 1 wei
uint256 computedMint = (tinyAmount * 1e18) / exchangeRate;
assertEq(computedMint, 0, &quot;Mint rounds to 0 — user gets zero AssetTokens&quot;);

uint256 computedFee = thunderLoan.getCalculatedFee(tokenA, tinyAmount);
assertEq(computedFee, 0, &quot;Fee rounds to 0 — flash loan is free&quot;);

// The deposit reverts because updateExchangeRate(0) can&#39;t increase the rate
vm.expectRevert(abi.encodeWithSelector(
    AssetToken.AssetToken__ExhangeRateCanOnlyIncrease.selector, exchangeRate, exchangeRate
));
thunderLoan.deposit(tokenA, tinyAmount); // Reverts — user&#39;s tokens never arrive
```

Test: `testRoundingErrors()` **— PASS**

RECOMMENDATION / MITIGATION

Add a minimum deposit threshold, a minimum fee floor, and dust tracking to prevent value loss from rounding.

```
<span class="gi">+ // Add minimum deposit requirement</span>
<span class="gi">+ uint256 public constant MINIMUM_DEPOSIT = 1e15; // 0.001 token</span>

<span class="gi">+ modifier revertIfBelowMinimum(uint256 amount) {</span>
<span class="gi">+     if (amount &lt; MINIMUM_DEPOSIT) revert ThunderLoan__AmountTooSmall();</
span>
<span class="gi">+     _;</span>
<span class="gi">+ }</span>

<span class="w"> </span> // Apply to deposit
<span class="w"> </span> function deposit(IERC20 token, uint256 amount)
<span class="gd">-     external revertIfZero(amount) revertIfNotAllowedToken(token) {</span>
<span class="gi">+     external revertIfZero(amount) revertIfBelowMinimum(amount)
revertIfNotAllowedToken(token) {</span>
<span class="w"> </span>     // ...
<span class="w"> </span> }

<span class="w"> </span> // Add minimum fee in getCalculatedFee
<span class="w"> </span> function getCalculatedFee(IERC20 token, uint256 amount) public view
returns (uint256 fee) {
<span class="w"> </span>     uint256 valueOfBorrowedToken = (amount *
getPriceInWeth(address(token))) / s_feePrecision;
<span class="w"> </span>     fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;
<span class="gi">+     if (fee &lt; 1e12) fee = 1e12; // Minimum fee floor</span>
<span class="w"> </span> }

<span class="gi">+ // Track accumulated dust</span>
<span class="gi">+ mapping(IERC20 =&gt; mapping(address =&gt; uint256)) public s_dust;</span>
```

MEDIUM

M-5: No Timelock on Any Owner Function

Severity: Medium — *Griefing*

DESCRIPTION

All `onlyOwner` functions execute instantly with no delay, no two-step process, and no user notice period. There is no timelock, no commit-reveal, no governance vote, and no multi-sig requirement. All three critical owner functions (`updateFlashLoanFee`, `setAllowedToken`, `_authorizeUpgrade`) are single-transaction operations.

IMPACT

Users cannot react to pending changes. There is no opportunity to withdraw before a detrimental fee increase or token removal. MEV bots can profit from fee changes (see M-3). Users have zero time to secure their positions.

PROOF OF CODE

This test demonstrates that a fee change from 0.3% to 50% executes in a single transaction with zero delay. There is no pending period, no notice, and no way for users to react.

```
// Fee changes from 0.3% to 50% — instantly, one transaction, zero delay
uint256 oldFee = thunderLoan.getFee(); // 3e15 = 0.3%
vm.prank(owner);
thunderLoan.updateFlashLoanFee(5e17); // 50%
assertEq(thunderLoan.getFee(), 5e17, &amp;quot;Fee changed — no timelock, no delay&amp;quot;);
```

Tests: `testNoTimelockOnOwnerActions()` — **PASS**, `testFrontrunningFeeChange()` — **PASS**

RECOMMENDATION / MITIGATION

Merge with C-4 mitigations. Apply the commit-reveal timelock pattern shown in M-3 to all three owner functions (`updateFlashLoanFee`, `setAllowedToken`, `_authorizeUpgrade`). Additionally, use a multi-sig wallet as the contract owner to distribute trust.

```
<span class="w"> </span> // Apply the same two-step pattern shown in M-3 to all owner functions:
<span class="w"> </span> // 1. proposeAction() — schedules the change with a deadline
<span class="w"> </span> // 2. executeAction() — executes only after the timelock expires

<span class="gi">+ // For updateFlashLoanFee — see M-3 Recommendation for full diff</span>
<span class="gi">+ // For setAllowedToken — see C-4 Recommendation for full diff</span>
<span class="gi">+ // For _authorizeUpgrade:</span>
<span class="gi">+ uint256 public s_upgradeDeadline;</span>
<span class="gi">+ address public s_pendingImplementation;</span>

<span class="gi">+ function proposeUpgrade(address newImplementation) external onlyOwner {</span>
<span class="gi">+     s_pendingImplementation = newImplementation;</span>
<span class="gi">+     s_upgradeDeadline = block.timestamp + OWNER_DELAY;</span>
<span class="gi">+     emit UpgradeProposed(newImplementation, s_upgradeDeadline);</span>
<span class="gi">+ }</span>

<span class="gi">+ function executeUpgrade() external onlyOwner {</span>
<span class="gi">+     require(s_upgradeDeadline != 0 &amp;&amp; block.timestamp &gt;=
s_upgradeDeadline, &quot;Timelock&quot;);</span>
<span class="gi">+     _authorizeUpgrade(s_pendingImplementation);</span>
<span class="gi">+     _upgradeToAndCall(s_pendingImplementation, &quot;&quot;);</span>
<span class="gi">+ }</span>
```

LOW

L-1: Missing Events for Critical State Changes

Severity: Low — *Contract fails to deliver promised returns*

DESCRIPTION

`updateFlashLoanFee()` modifies the `s_flashLoanFee` state variable but emits no event. Off-chain monitoring tools and users cannot track fee changes.

Location: `src/protocol/ThunderLoan.sol` lines 265–270, `ThunderLoanUpgraded.sol` lines 264–269

Tool Finding: Slither `events-maths`

RECOMMENDATION / MITIGATION

```
<span class="gi">+ event FlashLoanFeeUpdated(uint256 indexed oldFee, uint256 indexed newFee);</
span>

<span class="w"> </span> function updateFlashLoanFee(uint256 newFee) external onlyOwner {
<span class="w"> </span>     if (newFee &gt; s_feePrecision) revert ThunderLoan__BadNewFee();
<span class="gi">+     emit FlashLoanFeeUpdated(s_flashLoanFee, newFee);</span>
<span class="w"> </span>     s_flashLoanFee = newFee;
<span class="w"> </span> }
```

LOW

L-2: Events Missing `indexed` Fields

Severity: Low — *Contract fails to deliver promised returns*

9 instances across `AssetToken.sol`, `ThunderLoan.sol`, and `ThunderLoanUpgraded.sol`. Adding `indexed` to event parameters allows off-chain tools to efficiently filter and search for specific events.

Tool Finding: Aderyn L-4

Example fix:

```
<span class="gd">- event ExchangeRateUpdated(uint256 newExchangeRate);</span>
<span class="gi">+ event ExchangeRateUpdated(uint256 indexed newExchangeRate);</span>
```

LOW

L-3: Functions Marked `public` Should Be `external`

Severity: Low — *No value loss*

6 instances: `repay()`, `getAssetFromToken()`, `isCurrentlyFlashLoaning()` in both contracts. `external` is more gas-efficient because arguments are read from `calldata` rather than copied to `memory`.

```
<span class="gd">- function repay(IERC20 token, uint256 amount) public {</span>
<span class="gi">+ function repay(IERC20 token, uint256 amount) external {</span>
<span class="gd">- function getAssetFromToken(IERC20 token) public view returns (AssetToken) {</
span>
<span class="gi">+ function getAssetFromToken(IERC20 token) external view returns (AssetToken) {</
span>
<span class="gd">- function isCurrentlyFlashLoaning(IERC20 token) public view returns (bool) {</
span>
<span class="gi">+ function isCurrentlyFlashLoaning(IERC20 token) external view returns (bool) {</
span>
```

LOW

L-4: Empty Block in `_authorizeUpgrade()`

Severity: Low — *No value loss*

Both contracts have `_authorizeUpgrade()` with an empty body containing only the `onlyOwner` modifier. The empty block is misleading but functional (the modifier provides the access control).

```
<span class="gd">- function _authorizeUpgrade(address newImplementation) internal override
onlyOwner { }</span>
<span class="gi">+ function _authorizeUpgrade(address newImplementation) internal override
onlyOwner {</span>
<span class="gi">+     // Authorization enforced via onlyOwner modifier</span>
<span class="gi">+     // Add compatibility checks for newImplementation here in future</span>
<span class="gi">+ }</span>
```

LOW

L-5: Unused Custom Error

Severity: Low — *No value loss*

`ThunderLoan__ExhangeRateCanOnlyIncrease()` is defined in both contracts but never used. The actual exchange rate validation is in `AssetToken.sol` using `AssetToken__ExhangeRateCanOnlyIncrease`.

```
<span class="gd">- error ThunderLoan__ExhangeRateCanOnlyIncrease();</span>
```

LOW

L-6: PUSH0 Opcode Not Supported on All Chains

Severity: Low — *Contract fails to deliver promised returns (Immunefi)*

All 8 `.sol` files use `pragma solidity 0.8.20`, defaulting to the Shanghai EVM target that introduces `PUSH0`. This opcode is not supported on some L2 chains before the Dencun upgrade.

```
<span class="w"> </span> // In foundry.toml:
<span class="gi">+ evm_version = &quot;paris&quot;</span>
```

LOW

L-7: State Variables Written After External Calls

Severity: Low — *No value loss (subsumed by C-1)*

`reentrancy-benign` and `reentrancy-events` detections from Slither for `s_currentlyFlashLoaning[token]` writes after external calls. Addressed in **C-1** mitigations.

LOW

L-8: No Signature Replay Risk (Informational)

Severity: Low — *No value loss*

The protocol uses **zero** signature-based or meta-transaction functions: no EIP-2612 `permit()`, no `executeMetaTransaction()`, no ERC2771 trusted forwarder, no off-chain signatures. The `flashloan()` `params` bytes carry no nonce or expiry, but since no signatures are used, replay attacks via signature malleability are not applicable.

Test: `testNoSignatureReplay()` — **PASS**

LOW

L-9: Uninitialized Proxy State (Informational)

Severity: Low — *No value loss (deployment hygiene)*

Before `initialize()` is called, the proxy returns: `owner() = address(0)`, `getFeePrecision() = 0`, `getFee() = 0`, and `getCalculatedFee()` reverts (division by zero). The contract does call `_disableInitializers()` in the constructor, which prevents initialization of the implementation contract itself — only the proxy can be initialized.

Test: `testUninitializedProxy()` — **PASS**

Summary — Immunefi Referenced Classification

#	Finding	Severity	Immunefi Criteria	Tests
C-1	Reentrancy in `flashloan()`	**Critical**	Direct theft of user funds	3/3 ✔
C-2	Exchange Rate Miscalculation → Insolvency	**Critical**	Protocol insolvency	3/3 ✔
C-3	Fee-on-Transfer Tokens Break Accounting	**Critical**	Protocol insolvency	1/1 ✔
C-4	Centralization — Single Owner	**Critical**	Direct theft / Freezing of funds	3/3 ✔
C-5	Storage Collision on Upgrade	**Critical**	Protocol insolvency	1/1 ✔
H-1	Oracle Price Manipulation	**High**	Theft of unclaimed yield	2/2 ✔
H-2	Token Removal Traps LP Funds	**High**	Temporary freezing of funds	1/1 ✔
M-1	Missing Zero-Address Checks (all contracts)	**Medium**	Contract unable to operate / Fund burning	✔ @audit
M-2	Direct Donation Discrepancy	**Medium**	Griefing	1/1 ✔
M-3	Frontrunning Fee Changes (MEV)	**Medium**	Griefing / Theft of yield	1/1 ✔
M-4	Rounding Errors → Dust Loss / Zero Fees	**Medium**	Fails to deliver promised returns	1/1 ✔
M-5	No Timelock on Owner Functions	**Medium**	Griefing	2/2 ✔
L-1	Missing Events for Fee Updates	**Low**	Fails to deliver	—
L-2	Events Missing `indexed` Fields	**Low**	Fails to deliver	—
L-3	`public` Should Be `external`	**Low**	No value loss	—
L-4	Empty Block in `_authorizeUpgrade()`	**Low**	No value loss	—
L-5	Unused Custom Error	**Low**	No value loss	—
L-6	PUSH0 Not Supported	**Low**	Fails to deliver	—
L-7	State Variables After External Calls	**Low**	No value loss (subsumed)	1/1 ✔

#	Finding	Severity	Immunefi Criteria	Tests
L-8	No Signature Replay (Informational)	**Low**	No value loss	1/1 ✔
L-9	Uninitialized Proxy (Informational)	**Low**	No value loss	1/1 ✔

Running All Tests

```
# Deep security probe tests (8 tests across 8 attack vectors)
forge test --match-path test/unit/DeepSecurityProbeTest.sol -vvv
# Expected: 8/8 passing

# Static + dynamic exploit tests (8 tests)
forge test --match-path test/unit/ExploitTests.t.sol -vvv
# Expected: 8/8 passing

# Original unit tests
forge test --match-path test/unit/ThunderLoanTest.t.sol -vvv
# Expected: 7/7 passing

# All tests combined
forge test -vvv
# Expected: 25/25 passing
```

Report generated combining Slither (100 detectors), Aderyn static analysis, and Foundry-based dynamic exploit testing (16 test cases across 8 attack vectors plus static tool outputs). All severities classified per Immunefi bug bounty standard. Every finding follows the structure: Title, Description, Impact, Proof of Code (with explanation), Recommendation / Mitigation (with diff format showing exact code changes).